

Lecture 14: Model-Based RL. Part I: Basics and Planning

Prediction, imagination, and decision making

Petr Kuderov

Researcher at MIPT & AIRI

May 13, 2026

Motivation

Model-free RL learns from real interaction.

Every improvement requires experience a transition:

$$(s_t, a_t, r_t, s_{t+1})$$

This is often expensive:

- robotics trials are slow
- exploration can be unsafe
- sparse rewards waste episodes
- long horizons delay feedback
- simulation may be imperfect
- real data can be limited

Can the agent learn to predict consequences before acting?

Prediction Before Action

The basic idea of Model-Based RL:

Learn a model of the environment and use it for decision making.

Model-free RL:

- learn values / policy directly
- use real transitions
- react through experience

Model-based RL adds:

- learn predictive structure
- simulate possible futures
- plan or train in imagination

MBRL is about using prediction to improve control, not just about having a model.

What Is a Model?

By model we mean a model of the environment: transition and reward functions.

$$T(s, a) = p(s_{t+1} \mid s_t, a_t)$$

$$R(s, a, s') = p(r_t \mid s_t, a_t, s_{t+1})$$

In practice, a model can be:

- known or learned
- deterministic or stochastic
- tabular or neural
- exact or approximate
- state-/obs-space or latent-space
- one-step or sequence model

Sometimes an inverse transition model is [also] learned (*NB: regular one is called “forward model”*):

$$T_{\text{inv}}(s_t, s_{t+1}) = p(a_t \mid s_t, s_{t+1})$$

Do We Need a Perfect Simulator?

A common misconception:

better prediction → automatically better RL

While it's more like:

a model is good if it helps choose better actions

The model may ignore details that are irrelevant for control.

It may also predict visually plausible futures but still be bad for decision making.

MBRL needs control-relevant prediction, not necessarily realistic reconstruction.

Why Models Help

A learned model can be useful in several different ways.

Planning

Evaluate possible action sequences before executing them.

Exploration

Use prediction error or uncertainty to seek unknown states.

Synthetic experience

Generate additional transitions for value or policy learning.

Representation

Learn latent structure useful for control.

Lecture Roadmap

Right now we focus on foundations.

Main concepts:

- planning with known dynamics
- Monte Carlo Tree Search
- Dyna-style learning
- imperfect models

Practice:

- MCTS notebook
- Dyna in Taxi
- imperfect Dyna
- intrinsic motivation

The next lecture will cover modern applications and popular large-scale methods.

Planning with Known Dynamics



Planning vs Learning

Environment model is known $\rightarrow$ we can plan.

Planning means using a model internally instead of relying only on real interaction.

Planning for action selection: search over possible futures to choose the best next action.

$$s_t \rightarrow (a_t, a_{t+1}, \dots, a_{t+H})$$

Examples:

- tree search
- MCTS
- MPC / trajectory optimization

Planning for value improvement: use simulated transitions for Bellman-style updates:

$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a')$$

- value iteration
- prioritized sweeping
- Dyna planning updates

Lookahead Search

Starting from state s_t , unroll [all/many] possible trajectories using the model.

Solve explicitly which action to do next:

$$a_t^* = \arg \max_a \mathbb{E}[r_t + \gamma r_{t+1} + \dots + \gamma^H r_{t+H}]$$

The same trial-and-error search as in model-free RL.

Planning just trades real interaction for internal computation.

Bonus: *save-and-load* feature — can unroll many trajectories from any state.

While in real env we have to return to the state explicitly to unroll another trajectory.

Imaginary Rollouts for Action Value Estimate

An imaginary rollout is a trajectory generated by the model [and some policy].

$$s_t, a_t, r_t, s_{t+1}, a_{t+1}, r_{t+1}, \dots$$

Rollouts can be used to estimate action quality:

$$Q(s_t, a_t) \approx \mathbb{E} \left[\sum_{k=0}^H \gamma^k r_{t+k} \right]$$

The simplest strategy:

- sample many action sequences
- simulate their outcomes
- choose the best first action
- execute only one action
- observe real transition
- replan from the new state

Treat value estimate as information compression.

Receding Horizon / MPC Intuition

Planning far into the future is hard — the number of possible futures explodes, error accumulation, etc.
So we often plan only a short horizon, find and execute best action, and plan again.

plan → act → observe → replan

This is the basic idea behind Model Predictive Control.

Strength:

- corrects mistakes after each step
- robust to model errors

Weakness:

- can be myopic
- needs fast planning

Planning Needs Search

The number of action sequences grows exponentially.

If there are $|A|$ actions and horizon H :

$$\text{number of sequences} = |A|^H$$

For large problems, exhaustive search is impossible.

Good planning means searching selectively.

Monte Carlo Tree Search



Monte Carlo Tree Search

MCTS is planning by selective simulation.

Note: all possible future trajectories from a state s may be represented as a tree.

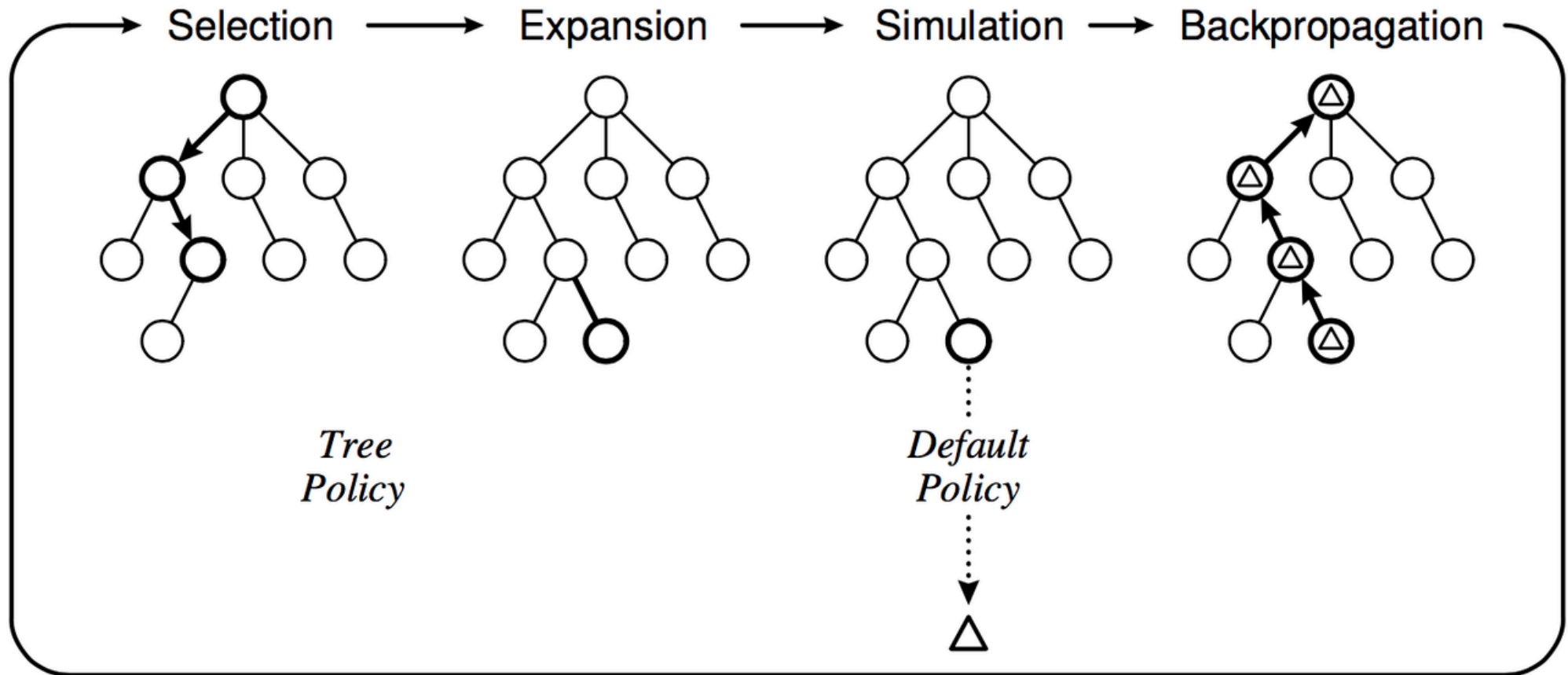
Instead of expanding the whole tree, it spends computation on promising branches.

Basic cycle:

- selection
- expansion
- rollout / simulation
- backup

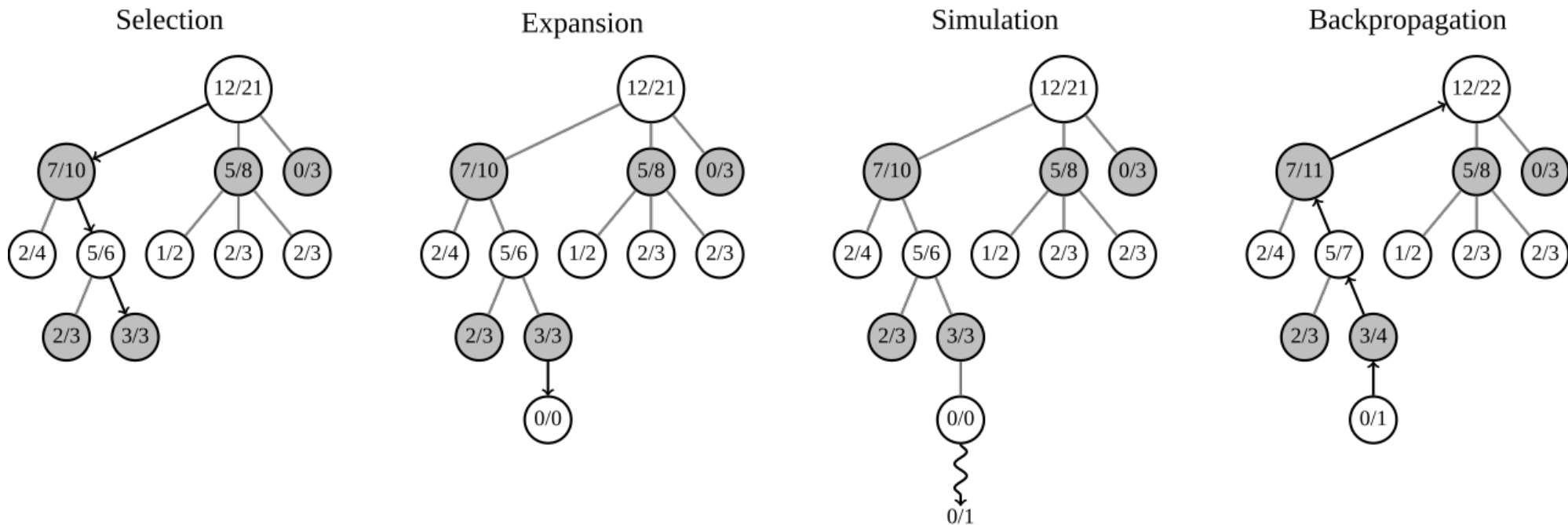
MCTS estimates action values by repeatedly imagining futures.

MCTS Scheme



MCTS aggregates information about state-action values in its nodes.

MCTS Example



Q-values are represented as two variables: visitation count N and sum of returns S_G :

$$Q(s, a) = \frac{S_G(s, a)}{N(s, a)} \dots \leftarrow \frac{S_G(s, a) + G}{N(s, a) + 1}$$

Selection: Where to Search?

MCTS needs to balance two motives.

Exploit

Search actions that already look good.

Explore

Search actions whose value is still uncertain.

A common rule is UCB-style selection:

$$a = \arg \max_a \left(Q(s, a) + c \sqrt{\frac{\log N(s)}{N(s, a)}} \right)$$

The exact constants and variants are less important here than the principle.

Backup: Learning From Imagined Futures

After a rollout, the simulated return is propagated back through the tree.

For visited state-action pairs:

$$Q(s, a) \leftarrow \text{average simulated return}$$

This gradually improves accumulated expectations near the root (=current state).

Instead of learning global solution (=policy) MCTS learns a local solution.

After taking an action in the real env, the selected sub-tree branch is kept for the next planning phase.

MCTS in Practice

MCTS is powerful when:

- model is available
- states and actions are discrete (or easy to discretize)
- simulation is fast and cheap
- long-term planning matters
- branching factor is manageable
- value estimates can guide search
- terminal rewards are informative
- re-planning is possible (=save-and-load is available)

Planning and Value Functions

Planning and value learning can help each other.

Simulated rollouts estimate local function:

$$Q(s, a)$$

A learned global value function for backups:

$$r + \gamma V(s')$$

Many modern systems combine search, learned value functions, and learned models.

Examples like AlphaZero and MuZero follow this logic.

Dyna: Models as Synthetic Experience



From Planning to Learning in Imagination

Model can also be used to generate extra training data.

Real transition:

$$(s, a, r, s')$$

Imagined transition:

$$(s, a, \hat{r}, \hat{s}')$$

Example: Dyna uses both real and imagined experience to update values or policies.

Sutton, 1990: Integrated architectures for learning, planning, and reacting based on approximating dynamic programming.

Dyna Architecture

Transitions from real experience:

- update value / policy
- update the model

Transitions from the model:

- sample past states and actions
- generate imagined transitions
- do extra RL updates

Note: one real transition can support many updates.

real experience → model update → many imagined updates

This is useful when:

- environment interaction is expensive
- the model is reasonably accurate
- tasks are structured
- old experience remains useful

Such model application improves so called “sample efficiency”

Tabular Dyna-Q

Initialize $Q(s, a)$ and $Model(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

(a) $S \leftarrow$ current (nonterminal) state

(b) $A \leftarrow \varepsilon$ -greedy(S, Q)

(c) Take action A ; observe resultant reward, R , and state, S'

(d) $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

(e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)

(f) Loop repeat n times:

$S \leftarrow$ random previously observed state

$A \leftarrow$ random action previously taken in S

$R, S' \leftarrow Model(S, A)$

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

Q-learning

Model Update

Planning Step

Dyna connects several separate ideas.

Planning:

- simulate futures
- backup values

Replay:

- reuse past states
- update from stored experience

Model learning:

- estimate dynamics
- estimate rewards

Imagination:

- create synthetic transitions
- train without acting

Many later MBRL methods are sophisticated descendants of this idea.

Practice: Dyna-Q in Taxi

Imperfect Models



The Model Is Usually Wrong

In real problems, the model is not known $\Rightarrow$ model is learned and approximate.

$$\hat{p}_{\theta}(s_{t+1}, r_t \mid s_t, a_t) \neq p(s_{t+1}, r_t \mid s_t, a_t)$$

Small model errors can be [and usually are] amplified by the usage.

A bad model can make the agent confidently learn the wrong thing.

Vicious circle: degrade model “trains” degrade policy to further collect degrade data.

Model Bias

Model-free learning has sampling noise.

Model-based learning adds model bias.

Real transition:

$$s' \sim p(\cdot | s, a)$$

noisy but grounded.

Imagined transition:

$$\hat{s}' \sim \hat{p}_{\theta}(\cdot | s, a)$$

cheap but biased.

The central MBRL trade-off: less real data, more dependence on model quality.

Compounding Error

One-step prediction error may look small.

Long rollouts can drift far away from reality.

$$s_t \rightarrow \hat{s}_{t+1} \rightarrow \hat{s}_{t+2} \rightarrow \dots \rightarrow \hat{s}_{t+H}$$

At every step, the model receives its own imperfect prediction as input.

This is why long imagined rollouts are dangerous.

The Planner Can Exploit the Model

Planning searches for high-return futures inside the learned model.

If the model has errors, the planner may discover unrealistic shortcuts.

“high return in the model” $\neq$ “high return in the environment”

The better the optimizer, the more it can exploit model mistakes.

Why Short Rollouts Often Help

A practical compromise:

use short imagined rollouts and frequent grounding in real data

This reduces compounding error while still improving sample efficiency.

Short rollout:

- less drift
- more grounded
- weaker planning

Long rollout:

- more foresight
- more bias
- more exploitation risk

Practice: imperfect Dyna in Taxi.

Exploration Through Prediction



Models Can Also Drive Exploration

A model tells us what the agent expects.

Unexpected transitions can signal novelty.

Prediction error:

$$e_t = \| s_{t+1} - \hat{s}_{t+1} \|$$

Intrinsic reward:

$$r_t^I = \beta e_t$$

The agent is rewarded for visiting states it cannot yet predict well.

Curiosity Intuition

External reward signals:

Was this useful for the task?

Intrinsic reward signals:

Did this teach me something new?

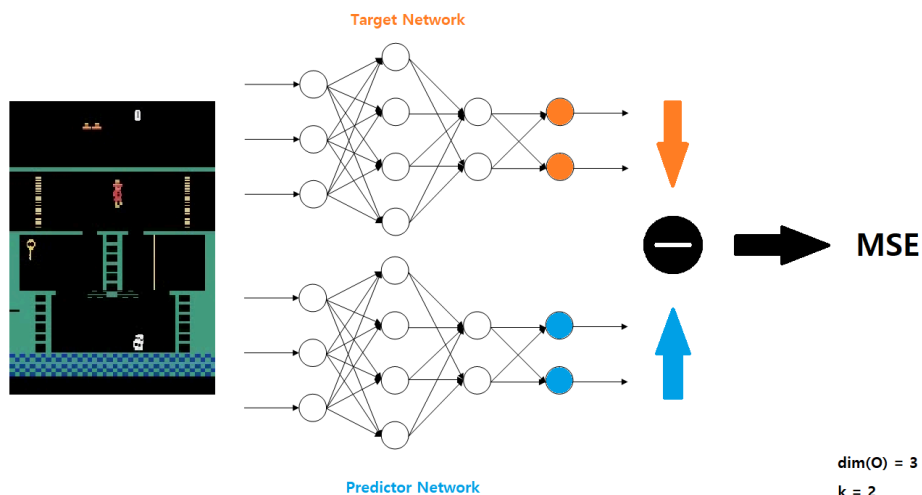
This can help in sparse-reward environments.

Practice: Dyna + intrinsic motivation in Taxi.

RND, ICM

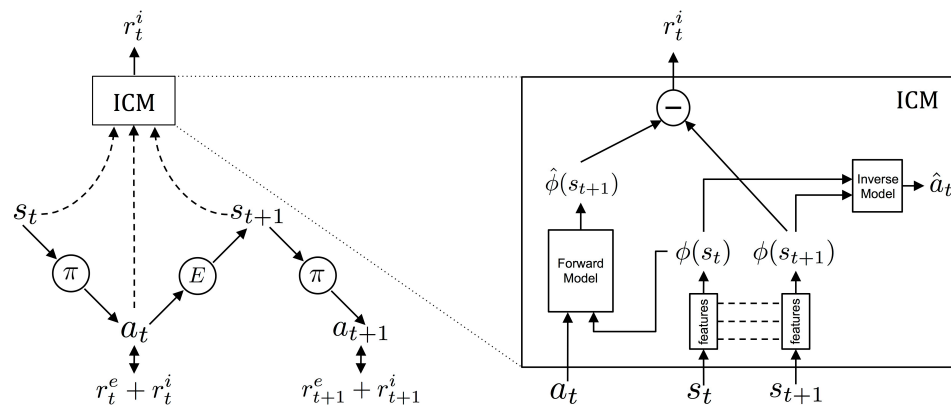
Random Network Distillation (RND):

- not exactly model-based, but mimics it
- env-proxy: some random frozen network $y(o)$
- model-proxy predicts the output
- prediction error — proxy for novelty



Intrinsic Curiosity Module (ICM):

- uses forward and inverse models
- inverse is used to learn latents
- forward is used to get novelty



Prediction Error Is Not Always Novelty

Prediction error can be high for several reasons.

Good reason:

- model does not know this region yet
- more data can reduce error

Bad reason:

- environment is inherently stochastic
- observation contains noise
- model class is too weak

Curiosity based on raw prediction error can chase noise.

See [this example](#).

Epistemic vs Aleatoric Uncertainty

Two kinds of uncertainty matter.

Epistemic

The model does not know because data is missing.

Can be reduced by exploration.

Aleatoric

The world is genuinely random.

Cannot be removed by more data.

Good exploration should target epistemic uncertainty.

Model Disagreement

One common proxy for epistemic uncertainty is ensemble disagreement.

Train several models:

$$\hat{p}_{\theta_1}, \hat{p}_{\theta_2}, \dots, \hat{p}_{\theta_K}$$

If they disagree, the state-action region is probably underexplored.

$$r_t^I \propto \text{disagreement}(\hat{s}'_1, \dots, \hat{s}'_K)$$

This idea appears in many exploration and uncertainty-aware MBRL methods.

Putting It Together



Four Uses of a Model

1. Planning

Use the model to select actions.

Example: MCTS, MPC.

2. Synthetic experience

Use the model to train values or policies.

Example: Dyna.

3. Exploration

Use model error or uncertainty as intrinsic motivation.

4. Representation

Use prediction to learn useful internal state.

Planning vs Learning With a Model

Planning use:

$$s_t \rightarrow \text{search} \rightarrow a_t$$

The model directly affects action choice.

Learning use:

$$\text{model} \rightarrow \text{imagined data} \rightarrow Q, \pi$$

The model improves training signal.

These are complementary, not mutually exclusive.

Known Model, Learned Model, Latent Model

Classical planning:

- model is given
- search is the main problem

Dyna:

- model is learned
- model generates updates

Modern world models:

- state is latent
- model learns representation
- policy trains in imagination

The next lecture will expand the modern world-model side.

Why Modern MBRL Uses Latent Models

Predicting raw observations can be unnecessary and hard.

Instead, learn an internal state:

$$s_t \rightarrow z_t = \varphi(s_t)$$

and predict in latent space:

$$p_{\theta}(z_{t+1} \mid z_t, a_t)$$

The model should preserve what matters for future control.

Examples to be covered later: PlaNet, Dreamer, MuZero-like systems.

Interaction w/ HRL

MBRL and HRL meet naturally.

A model enables abstract planning:

HRL question:

- what subgoal should I pursue?
- what skill should I execute?

MBRL question:

- what future states are possible?
- which future is worth pursuing?

Hierarchy often needs prediction. Prediction often reveals hierarchy.

Some directions:

- multistep prediction forward models
- 2-phase episodes: plan/exploit then explore

Limitations of Model-Based RL

MBRL is powerful, but fragile.

Common problems:

- model bias
- compounding error
- planner exploits model flaws
- stochasticity
- partial observability
- expensive planning
- hard uncertainty estimation
- representation mismatch

The model is a tool, not a guarantee.